



Pontifícia Universidade Católica de Campinas

Engenharia de Computação

Projetos de Sistemas Operacionais

Relatório Final do Projeto Toytrace

Semana 6: Polimento, Testes e Documentação Final

Professor: Prof. Msc. Lucas Reis

Integrantes do grupo

Nome	RA
Bruno Vinicius Selingard	24004238
João Vitor Bombo Guimarães	24018551
Luccas Gomes Zibordi	24007138
Miguel de Castilho Gengo	24007009

Repositório:

https://github.com/LuccasZibordi/Projeto_toytrace_grupo1

Campinas
2026

1 Introdução

O projeto *Toytrace* teve como objetivo desenvolver um pequeno monitor de chamadas de sistema em Linux, inspirado no funcionamento do *strace*. A proposta consistiu em executar um programa alvo como processo filho, controlar sua execução por meio de *ptrace*, identificar entradas e saídas de chamadas de sistema, ler registradores, montar eventos e apresentar uma saída legível ao usuário.

Na Semana 6, o foco principal foi a finalização do projeto. As atividades se concentraram no polimento do código, na correção de problemas restantes, na limpeza de mensagens temporárias de depuração, na execução dos testes finais e na elaboração da documentação final do trabalho.

2 Estrutura Geral do Projeto

O sistema foi organizado em módulos com responsabilidades bem definidas. A entrada do programa ocorre em `main.c`, responsável por acionar a interface de linha de comando e iniciar o fluxo principal. O arquivo `cli.c` interpreta os argumentos informados pelo usuário, como o comando `trace` e a opção `-raw-events`.

A parte central do monitoramento fica em `trace_runtime.c`, onde são realizadas as etapas de criação do processo filho, configuração do rastreamento, espera por paradas de `syscall` e leitura dos registradores. A partir desses dados, o projeto preenche uma estrutura de evento contendo o número da `syscall`, argumentos, retorno e estado de entrada ou saída.

Os arquivos em `src/student` concentram a parte implementada pelos alunos. O `pairer.c` realiza o pareamento entre entrada e saída da mesma `syscall`, enquanto o `formatter.c` transforma o evento completo em uma linha compreensível para o terminal.

3 Commits e Evolução Semanal

Semana	Descrição das atividades realizadas
Semana 1	Análise inicial do projeto, execução dos comandos básicos, estudo da estrutura dos arquivos e compreensão do fluxo principal do programa.
Semana 2	Implementação da criação do processo alvo com <code>fork</code> , preparação do filho com <code>PTRACE_TRACEME</code> , parada inicial com <code>SIGSTOP</code> e início do pareamento de syscalls.
Semana 3	Implementação do loop de rastreamento com <code>PTRACE_SYSCALL</code> , configuração de opções com <code>PTRACE_O_TRACESYSGOOD</code> e detecção das paradas de syscall.
Semana 4	Leitura dos registradores com <code>PTRACE_GETREGS</code> e preenchimento da estrutura <code>syscall_event</code> com número da syscall, argumentos e retorno.
Semana 5	Implementação da saída legível, tratamento de syscalls principais e melhoria na apresentação das chamadas observadas.
Semana 6	Revisão final, limpeza do código, correção de detalhes, execução de testes com <code>make test</code> e preparação do relatório final.

Tabela 1: Resumo da evolução semanal do projeto.

4 Divisão do Trabalho

A divisão do trabalho foi feita de forma semanal, acompanhando as etapas propostas para o projeto. Na Semana 1, todos os integrantes participaram da revisão inicial dos arquivos, entendimento do funcionamento geral do projeto e elaboração da documentação preliminar. Nas semanas seguintes, cada integrante ficou associado a funções ou trechos específicos do código, contribuindo para a construção gradual do *Toytrace*.

Tabela 2: Divisão das atividades e funções implementadas entre os integrantes do grupo.

Integrante	Semanas, funções e contribuições principais
Bruno Vinicius Se- lingard	Semana 1: participou da revisão inicial do projeto, análise dos arquivos fornecidos e auxílio na elaboração do relatório/documentação da semana. Semana 2: ficou associado ao apoio na função <code>wait_for_initial_stop()</code>, auxiliando na validação da parada inicial do processo filho após o uso de <code>SIGSTOP</code>. Semana 3: ficou associado à função <code>configure_trace_options()</code>, responsável pela configuração das opções do <code>ptrace</code>, especialmente o uso de <code>PTRACE_O_TRACESYSGOOD</code>. Semana 5: contribuiu com dois trechos da etapa de formatação: tratamento da syscall <code>read</code> e apoio na saída genérica para chamadas ainda não tratadas de forma especial. Semana 6: participou da revisão final, execução dos testes, limpeza de mensagens temporárias e conferência do comportamento geral do programa.
João Vitor Bombo Guimarães	Semana 1: participou da revisão inicial do projeto, estudo da estrutura dos arquivos e apoio na documentação da primeira semana. Semana 2: ficou associado ao apoio na função <code>launch_tracee()</code>, relacionada à criação do processo filho com <code>fork()</code> e preparação inicial para o rastreamento. Semana 3: ficou associado à função <code>resume_until_next_syscall()</code>, utilizando <code>PTRACE_SYSCALL</code> para permitir que o processo monitorado avançasse até a próxima parada de syscall. Semana 4: contribuiu com a chamada de <code>PTRACE_GETREGS</code> dentro do fluxo de <code>trace_program()</code>, etapa responsável por obter os registradores do processo filho parado. Semana 5: contribuiu com dois trechos da formatação final: tratamento da syscall <code>write</code> e apoio na exibição do valor de retorno das chamadas de sistema.

Integrante	Semanas, funções e contribuições principais
Luccas Gomes Zibordi	Semana 1: participou da análise inicial do projeto, identificação do fluxo principal do código e apoio na elaboração da documentação da semana. Semana 2: ficou associado à função <code>student_pair_syscall()</code>, responsável por parear os eventos de entrada e saída da mesma syscall, juntando os argumentos capturados na entrada com o retorno obtido na saída. Semana 3: ficou associado à função <code>wait_for_syscall_stop()</code>, auxiliando na identificação das paradas de syscall e na diferenciação entre término do processo, sinais e eventos válidos de rastreamento. Semana 4: ficou associado à função <code>fill_event_from_regs()</code>, responsável por preencher a estrutura <code>syscall_event</code> com o número da syscall, retorno e argumentos a partir dos registradores <code>orig_rax</code>, <code>rax</code>, <code>rdi</code>, <code>rsi</code>, <code>rdx</code>, <code>r10</code>, <code>r8</code> e <code>r9</code>. Semana 6: participou da revisão final do código, execução de testes, correção de detalhes restantes e organização do relatório final.
Miguel de Castilho Gengo	Semana 1: participou da revisão inicial, estudo dos arquivos fornecidos e apoio na construção do relatório/documentação da semana. Semana 2: ficou associado ao apoio na validação do funcionamento de <code>student_pair_syscall()</code>, verificando o comportamento do pareamento entre entrada e saída das syscalls. Semana 3: ficou associado ao apoio no loop principal de <code>trace_program()</code>, acompanhando a alternância entre entrada e saída de syscall durante o rastreamento. Semana 4: ficou associado à função <code>student_debug_raw_event()</code>, utilizada para exibir eventos crus no modo <code>-raw-events</code>, auxiliando na validação dos dados capturados dos registradores. Semana 5: contribuiu com dois trechos da saída final: tratamento da syscall <code>execve</code> e tratamento da syscall <code>exit_group</code>, melhorando a legibilidade do resultado apresentado ao usuário.

5 Desafios Encontrados

Um dos principais desafios foi compreender o fluxo de execução entre processo pai e processo filho. O uso de `fork`, `ptrace`, `waitpid` e sinais exigiu atenção, pois pequenas diferenças no estado do processo alteravam completamente o comportamento do rastreamento.

Outro ponto importante foi entender que cada chamada de sistema gera duas paradas: uma na entrada e outra na saída. Por isso, foi necessário manter um estado temporário para guardar os argumentos capturados na entrada e combiná-los com o retorno obtido na saída.

Também houve dificuldade inicial na interpretação dos registradores da arquitetura `x86_64`. O número da `syscall` é obtido por `orig_rax`, o retorno por `rax`, e os argumentos pelos registradores `rdi`, `rsi`, `rdx`, `r10`, `r8` e `r9`. Essa etapa foi essencial para transformar eventos crus em informações úteis.

6 Aprendizados

O projeto permitiu compreender, de forma prática, como um sistema operacional interage com processos em execução. A implementação mostrou que chamadas de sistema são a ponte entre programas em modo usuário e serviços oferecidos pelo kernel, como leitura, escrita, abertura de arquivos e finalização de processos.

Além disso, o grupo aprendeu a importância de dividir responsabilidades em módulos, testar cada etapa separadamente e manter commits organizados. O desenvolvimento também reforçou conceitos de processos, sinais, registradores, chamadas de sistema, depuração e automação de testes com `make`.

7 Melhorias Futuras

Como melhorias futuras, o projeto poderia incluir uma formatação mais completa para outras `syscalls`, filtros por nome de chamada de sistema, contagem estatística das chamadas executadas e suporte a mais arquiteturas além de Linux `x86_64`. Também seria possível melhorar a leitura de strings da memória do processo filho para apresentar caminhos de arquivos e argumentos de execução com maior clareza.

8 Conclusão

Ao final da Semana 6, o projeto atingiu seu objetivo principal: construir uma versão didática de um monitor de chamadas de sistema. O trabalho permitiu acompanhar

desde a criação do processo monitorado até a produção de uma saída legível das syscalls executadas. A experiência consolidou conceitos fundamentais de Sistemas Operacionais e demonstrou, na prática, como ferramentas semelhantes ao *strace* podem ser construídas a partir de mecanismos oferecidos pelo Linux.